```
            handleCommands(state, command, sharding, context, timer),    ❸
          eventHandler = handleEvents)
            ...                                                          ❹
  }}}
  ...
}
```

❶ Timer scheduler used later to start the countdown

❷ Sets up the initial state

❸ Command handler using sharding, the context, and the timer

❹ Taggers, snapshot strategy, and restarting strategy

Let’s see how these commands and states work together.

Handling Commands: Open

In the uninitialized state, the only valid action is to open. Once you are in the `OpenState`, all commands are accepted, such as confirming funds or the market, settling, or closing. Regardless of the state, you can request to cancel the bet. However, a request can also be rejected. For example, if you try to validate or settle a bet after it has been closed, it will be rejected. Finally, there is an `invalid` option that handles all other combinations of commands that are not explicitly handled; it’s like saying you can’t select something if it’s not on the menu.

Listing 12.7 Entity constructor and handling commands

```
object Bet {
  ...

  def handleCommands(
      state: State,
      command: Command,
      sharding: ClusterSharding,
      context: ActorContext[Command],
      timer: TimerScheduler[Command]): Effect[Event, State] = {
    (state, command) match {
```